

경제 슷폼 트렌드 챗봇 팀 개발 계획

Oct 5, 2026 · @jt

프로젝트 개요

로그인한 경제 슷폼 제작자가 질문하면, 최근과 과거 트렌드를 비교해 오늘 올릴 슷폼 주제를 추천하는 FastAPI 챗봇입니다.

- **문제 정의:** AI·경제 정보가 너무 많아 무엇이 지금 뜨는 주제이고 무엇이 이미 지난 주제인지 구분하기 어렵다.
- **타겟 사용자:** 유튜브 경제·AI 슷폼 제작자
- **핵심 원칙:** LLM은 오늘의 이슈를 모르므로, 네이버 API로 수집한 트렌드 데이터를 프롬프트에 넣어 답한다.

모드	사용자 질문	챗봇이 하는 일
Q1	오늘 무슨 슷폼 올려야 해?	오늘 핫이슈 + 최근/과거 추이 비교 → 오늘 주제와 내용 정리. 채널 운영 중이면 과거 업로드 주제를 대화로 받아 연결 주제 제안(Q2 통합)
Q4	이 주제 지금 올려도 돼?	검색 추이·언급량으로 상승/정점/하락 판정 → 지금 올리기 / 기다리기 / 각도 바꾸기
Q5	너무 흔한데 다른 각도 없을까?	반전·비교·논쟁·정보·경험형으로 재해석해 주제 5~10개 생성, 1개 추천
Q6	어제 영상 다음 편은?	이전 주제의 파생 키워드로 1→2→3편 시리즈 구조와 오늘 올릴 편 추천

개발 범위

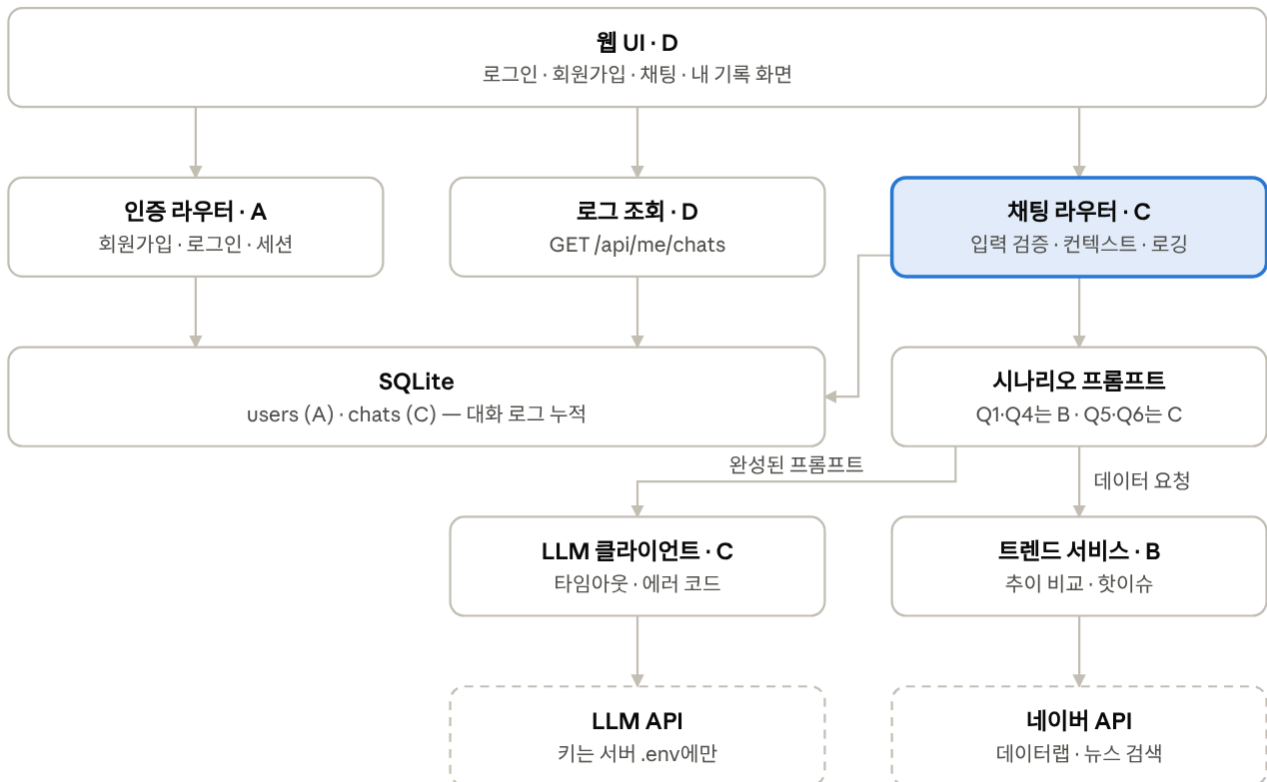
과제 필수 요구사항만 충족하도록 범위를 최소화했습니다. 단, 팀원별 커밋 10회 이상은 필수이므로 작업은 잘게 나눕니다.

구분	포함	제외
외부 데이터	네이버 데이터랩(검색 추이), 네이버 뉴스 검색 — 키 1개	YouTube API, 캐시 테이블, 스케줄러
시나리오	Q1·Q4·Q5·Q6 프롬프트 템플릿, <code>mode</code> 로 분기	Q2 별도 구현(→ Q1에 통합), LLM 의도 분류

구분	포함	제외
DB	users , chats 2개 테이블	채널 프로필 테이블
로그 확인	GET /api/me/chats , scripts/check_logs.sql	관리자 화면
인증	세션 쿠키 (Starlette SessionMiddleware)	JWT
프론트	Jinja2 템플릿 + 순수 JS	React 등 프레임워크
배포	VM 1대 (uvicorn + systemd)	컨테이너, CI/CD

시스템 구조

질문은 채팅 라우터에서 트렌드 데이터를 담아 AI로 가고 DB에 남는다



점선 박스 = 외부 API · 강조 박스 = 핵심 파이프라인 · 이름 옆 알파벳 = 담당 팀원

시스템 구조 · 컴포넌트와 담당자

채팅 라우터가 시나리오 프롬프트에 트렌드 데이터를 담아 LLM을 호출하고, 결과를 chats 테이블에 저장한 뒤 화면에 돌려줍니다.

```

app/
├─ main.py, config.py, database.py  A
├─ models/      user.py (A) · chat.py (C)
├─ routers/     auth.py (A) · chat.py (C) · logs.py, pages.py (D)
├─ services/    trend.py (B) · llm.py (C)
│   └─ scenarios/ q1.py, q4.py (B) · q5.py, q6.py (C)
├─ templates/, static/                D
scripts/check_logs.sql                D
.env.example, .gitignore, README.md

```

DB 구조 (SQLite)

테이블은 2개이며, `chats.user_id` 로 사용자별 대화 로그를 조회·추적합니다.

users — 담당 A

필드	타입	설명
id	INTEGER PK	사용자 식별
email	TEXT UNIQUE	로그인 ID
password_hash	TEXT	bcrypt 해시
created_at	DATETIME	가입 시각

chats — 담당 C

필드	타입	설명
id	INTEGER PK	대화 ID
user_id	INTEGER FK → users.id	사용자 식별 (필수 추적 필드)
mode	TEXT	q1 / q4 / q5 / q6 / free
question	TEXT	사용자 질문 (필수)
answer	TEXT	AI 응답, 실패 시 NULL (필수)
status	TEXT	success / timeout / error
created_at	DATETIME	생성 시각 (필수)

관계: users 1 — N chats

API 명세와 공통 인터페이스

첫날 이 계약만 맞추면 B와 C는 서로를 mock으로 대체해 병렬 개발할 수 있습니다.

메서드	경로	로그인	설명	담당
POST	/api/auth/signup	X	회원가입	A
POST	/api/auth/login	X	로그인 (세션 발급)	A
POST	/api/auth/logout	O	로그아웃	A
POST	/api/chat	O	질문 → AI 응답	C
GET	/api/me/chats	O	내 대화 로그 조회	D
GET	/, /login, /signup, /history	화면별	웹 페이지	D

POST /api/chat

```
// 요청
{"mode": "q4", "message": "금리 인하 주제 지금 올려도 돼?"}

// 성공 200
{"chat_id": 987, "answer": "검색량이 2주째 상승 중이라 지금 올리기를 추천해요..." }

// 실패 504
{"error": "AI_TIMEOUT", "message": "현재 응답이 지연되고 있어요. 잠시 후 다시 시도해 주세요."}

// 입력 오류 422
{"error": "INVALID_INPUT", "message": "질문은 1~500자로 입력해 주세요."}
```

GET /api/me/chats?limit=20

```
[{"id": 987, "mode": "q4", "question": "...", "answer": "...", "status": "success", "created_at": "2026-10-05T15:50:00"}]
```

내부 함수 계약

```
# B 제공 (services/trend.py)
get_hot_issues(limit: int = 10) -> list[dict]           # [{"title", "link",
"pub_date"}]
compare_periods(keyword: str) -> dict                  # {"recent", "past",
"trend": "rising"|"peak"|"falling"}

# C 제공 (services/llm.py)
async def ask_llm(system: str, messages: list[dict]) -> str  # 타임아웃 시
AITimeoutError

# 시나리오 공통 (services/scenarios/*.py)
async def build_prompt(message: str) -> str             # 트렌드 데이터를 넣은 시스
템 프롬프트 반환
```

역할 분담

기능 단위로 4명이 나누며, 체크 항목 1개가 커밋 1회 이상이 되도록 쪼갬습니다. 이름 칸은 팀원 확정 후 채워 주세요.

팀 원	담당 기능	브랜치	소유 파일
A	프로젝트 기반 + 인증	feature/auth	main.py, config.py, database.py, models/user.py, routers/auth.py
B	트렌드 데이터 + Q1·Q4	feature/trend	services/trend.py, scenarios/q1.py, q4.py
C	챗봇 파이프라인 + Q5·Q6	feature/chat	models/chat.py, routers/chat.py, services/llm.py, scenarios/q5.py, q6.py
D	UI + 로그 조회 + 배포· 문서	feature/ui	templates/, static/, routers/logs.py, scripts/, README.md

A — 프로젝트 기반 + 인증

- ☐ FastAPI 프로젝트 골격 (첫날 develop 머지)
- ☐ .env.example, .gitignore, config.py
- ☐ SQLite 연결, Base 모델
- ☐ User 모델
- ☐ 회원가입 API
- ☐ 비밀번호 해싱 (passlib/bcrypt)

- ☐ 로그인·로그아웃 (세션 쿠키)
- ☐ `require_login` 의존성 (비로그인 접근 차단)
- ☐ 회원가입 입력 검증 (이메일 형식, 비밀번호 길이, 중복 가입)
- ☐ 인증 테스트 + README 실행·환경 변수 섹션

B — 트렌드 데이터 + Q1·Q4

- ☐ 네이버 API 공통 클라이언트 (키는 env, 타임아웃 설정)
- ☐ 데이터랩 검색어 추이 조회
- ☐ 뉴스 검색 조회
- ☐ `get_hot_issues()` 오늘의 경제 이슈
- ☐ `compare_periods()` 최근 vs 과거 비교
- ☐ 상승/정점/하락 판정 로직
- ☐ 외부 API 실패 시 빈 데이터로 대체 + 로그
- ☐ 개발용 mock 데이터
- ☐ Q1 프롬프트 (Q2 채널 맥락 포함)
- ☐ Q4 프롬프트 (지금 / 기다리기 / 각도 바꾸기)

C — 챗봇 파이프라인 + Q5·Q6

- ☐ Chat 모델
- ☐ LLM 클라이언트 (서버 측 호출, 타임아웃)
- ☐ `POST /api/chat` 기본 흐름
- ☐ 이벤트 로깅 (`request_received`, `ai_call_start`, `ai_call_success/fail`, `db_save_success/fail`)
- ☐ 최근 5개 Q/A 컨텍스트 구성
- ☐ 질문 입력 검증 (빈 값 차단, 500자 제한)
- ☐ `mode` 별 시나리오 분기
- ☐ `AI_TIMEOUT / AI_ERROR` 에러 응답, 서버 비정상 종료 방지
- ☐ Q5 프롬프트 (5가지 각도로 5~10개 생성)
- ☐ Q6 프롬프트 (1→2→3편 시리즈)

D — UI + 로그 조회 + 배포·문서

- ☐ 베이스 템플릿, 공통 스타일
- ☐ 로그인·회원가입 화면
- ☐ 채팅 화면 (같은 화면에 응답 표시)
- ☐ 모드 선택 버튼 (Q1/Q4/Q5/Q6)
- ☐ 에러 메시지 표시, 응답 대기 표시

- ☐ GET /api/me/chats API
- ☐ 내 기록 화면
- ☐ scripts/check_logs.sql
- ☐ VM 배포 (uvicorn + systemd, 외부 접속 확인)
- ☐ README 통합 (개요, 구조, API, ERD, 팀 역할·개인별 작업 요약)

협업 규칙

feature/* → develop → main 순서로, 모든 머지는 PR로만 합니다.

```
main      배포용. develop에서만 PR로 머지
develop   통합 브랜치
feature/auth · feature/trend · feature/chat · feature/ui
```

커밋 컨벤션 — type(scope): 설명

type	용도	예시
feat	기능 추가	feat(chat): 최근 5개 대화 컨텍스트 적용
fix	버그 수정	fix(chat): 타임아웃 시 500 반환되던 문제 수정
docs	문서	docs: README API 명세 추가
refactor	동작 변화 없는 구조 개선	refactor(trend): 네이버 클라이언트 분리
chore	설정·의존성	chore: requirements.txt 추가
test	테스트	test(auth): 로그인 실패 케이스 추가

PR 규칙

- 머지 방식은 **Create a merge commit**만 사용합니다. Squash merge는 개인 커밋 수를 지워 버립니다.
- 1인 1브랜치여도 PR은 기능 묶음마다 2~3번 나눠 올립니다. 마지막에 PR 1개만 있으면 기능 단위 작업 흔적이 약해 보입니다.
- PR마다 팀원 1명 이상 리뷰 승인 후 머지합니다.
- 작업 시작 전 `git pull origin develop` 으로 최신 develop을 받아옵니다. `main.py` 와 `models/` 는 충돌이 잦습니다.
- `.env` 는 절대 커밋하지 않습니다. 새 키가 생기면 `.env.example` 에 이름만 추가합니다.

진행 순서와 요구사항 체크리스트

A의 골격 머지가 모든 작업의 출발점이고, 배포 URL은 평가 전에 외부 접속으로 반드시 확인합니다.

1. **세팅**: A가 골격을 develop에 머지, API 계약 합의, 각자 네이버·LLM API 키 발급
2. **개발**: B·C는 mock으로 서로 대체해 병렬 진행, D는 화면부터 제작
3. **통합**: 시나리오별 끝까지 연결, 타임아웃·빈 입력·비로그인 접근 테스트
4. **배포·문서**: VM 배포, 외부 접속 확인, README 완성, Git 이력과 역할 요약 대조

제출 전 체크리스트

- ☐ 회원가입·로그인 동작, 비로그인 시 챗봇 사용 불가
- ☐ 질문 입력 후 같은 화면에 응답 표시
- ☐ AI 호출은 서버에서만, 키는 `.env` 로 관리
- ☐ 최근 N개 대화 컨텍스트 적용
- ☐ 대화 로그 DB 저장 (`user_id`, `created_at`, `question`, `answer`)
- ☐ 사용자 기준 로그 조회 (`/api/me/chats` , `check_logs.sql`)
- ☐ 서버 로그 4종: 요청 수신 / AI 호출 / AI 응답·실패 / DB 저장 성공·실패
- ☐ AI 타임아웃 시 서버가 죽지 않고 오류 안내 반환
- ☐ 입력 검증 1개 이상 (빈 입력, 길이 제한)
- ☐ 외부 네트워크에서 서비스 URL 접속 가능
- ☐ `main/develop/feature` 브랜치, PR 머지 기록
- ☐ 팀원별 유의미한 커밋 10회 이상
- ☐ README: 개요, 구조, API 명세, ERD, 배포·환경 변수, 팀 역할·개인별 작업 요약
- ☐ `.env.example` 제공, `.gitignore` 에 `.env` 포함